

Submission 01: Summary of the project plan

1. Intro to project

The aim of this project is to create a C++ program that reads different configurations of a reverse 2048 game. This will then be played by a simple algorithm (Greedy algorithm) and a complex algorithm (Monte Carlo algorithm) simultaneously. The program must then store the results of each game, thus determining which algorithm most efficiently beats the game.

2. Algorithms

The first algorithm used will be a simple algorithm called a **greedy algorithm**. Greedy algorithms solve problems by making the best choice at each immediate step, without considering future problems or setbacks that may arise from the choice made. We have chosen to make use of this algorithm as sometimes the best strategy in beating a game like reverse 2048 is to focus solely on what move is best at the current moment, as it can be hard to predict where the game will generate the next tile and how it will affect the game.

The second algorithm used will be a complex algorithm called the **Monte Carlo Tree Search Algorithm**. The Monte Carlo Tree search algorithm will model the probability of different algorithms using random sampling of the game. Multiple possible outcomes will then be generated and an average result will be calculated, leading to a predictive probability-based play. It is one of the most common types of algorithms used in online game play, thus a good fit for the project problem.

3. Anticipated challenges and constraints, and how to address them

1. Lack of complex coding background

This project will be a challenge as we have yet to learn this level of coding in our lectures, thus most of the required techniques will have to be self-taught. This will be addressed by learning the provided Ulwazi course material on coding in C++ and by using additional learning resources such as C++ coding tutorials specific to the task at hand.

2. Time Constraints

This is a potential problem due to differing schedules and lack of time to practise and learn the techniques needed to complete this project. This will be addressed by allocating specific slots of time each week where we can meet up and work on the project, so that we do not need to rush in the last few weeks before submission.

4. Brief overview of code structures

Functions

- **BoardSetup(HighestTile,BoardGrid)** : generates the board grid using the BoardGrid value and stores the highest tile value possible to appear on the board from the HighestTile value.
- **Movement(Direction)** : Moves the tiles on the board in the direction given by the Direction value and combines the same tiles whilst moving the rest in the direction given.
- **Results** : stores the results of the games played by the different algorithms in the format specified for the project.